FORMAL PROOF MAINTENANCE

Fewer assumptions, shorter internal arguments

Three further Lean proof simplifications

for `DiophantineConsistency`

Prepared for Vladimir Reshetnikov

26 October 2026

Proof and patch baseline

`f00ba53b331833c6cb64707bd90e48489e5610f5`

Closing source and exclusion audit

`60fcb2a69b7f1fdd4c3592ea5f164223cad50abe`

https://github.com/VladimirReshetnikov/ProveIt

Abstract

This report treats all recommendations in the forty-three prior simplification packets at the closing audit as already available, including optional and unimplemented recommendations. Three residual opportunities survive the resulting nonduplication screen. First, the positive source equations themselves imply the first port's positivity and a stronger scale bound, so callers need not pass that positivity guard independently. Second, exact binary rotation depends only on a partial-bit phase and a cyclic whole-digit displacement; comparing these parameters removes Boolean hypotheses from a recently added rephasing proof. Third, internal Euclidean-division uniqueness can use separation of quotient intervals, avoiding the construction of a second successor-gap witness.

The accompanying Lean kernels, concrete source-row client, candidate patches, source manifests, and compiler records make these proposals reviewable. The report distinguishes checked lemmas from candidate integration and does not present isolated checks as a complete repository build. Two proposals strengthen interfaces; the division proposal mainly simplifies the mathematical argument. No aggregate line-reduction claim is made against an imaginary checkout containing every earlier proposal.

Scope: proof structure and assumptions. The printed Diophantine equation and the mathematical consistency claim are not changed.

Contents

1 Findings and recommended order **1**

2 Baseline, evidence, and the exclusion rule **1**
2.1 What was inspected . . . 1
2.2 The counterfactual baseline . . . 2
2.3 Examples excluded by the audit . . . 2
2.4 Closing refresh: forty-three packets . . . 2

3 Recover the positive port guard from the source equations **3**
3.1 The existing dependency . . . 3
3.2 A stronger guard-recovery theorem . . . 4
3.3 The checked concrete source client . . . 4
3.4 How to integrate it after the earlier proposals . . . 5
3.5 Why the assumptions cannot simply disappear everywhere . . . 5
3.6 Novelty boundary and expected value . . . 5

4 Compare rotation parameters instead of proving digit bounds **6**
4.1 What the definition actually reads . . . 6
4.2 Exact periodicity, including partial-bit phases . . . 6
4.3 The present rephase client . . . 7
4.4 The distinction that prevents an unsound shortcut . . . 7
4.5 Novelty boundary and accounting . . . 7

5 Prove internal division uniqueness by interval separation **8**
5.1 The exact residual proof . . . 8
5.2 The interval argument . . . 8
5.3 What the delivered replacement removes . . . 8
5.4 Why host-natural division is not a replacement . . . 9
5.5 Novelty boundary and measured value . . . 9

6 Validation and interpretation of the artifacts **9**
6.1 What the checks establish . . . 10
6.2 No claimed end-to-end consistency advance . . . 10

7 Integration plan and stopping criteria **10**
7.1 Stage 1: source guard recovery . . . 10
7.2 Stage 2: rotation congruence . . . 11
7.3 Stage 3: interval uniqueness . . . 11

8 Archive guide and reproducibility **11**

Selected sources and nearest prior proposals **12**

1 Findings and recommended order

The most useful new change is to *derive redundant guards at the point where the literal source equations become available*. It strengthens the source-facing API and removes a dependency that several later proofs currently thread through their signatures. The rotation change is small and sharply bounded. The division change is a mathematical cleanup with a small local line saving; its main value is a simpler invariant.

Order	Proposal	Concrete benefit	Accounting rule
1	Recover the port guard from the source	Derive $\beta > 0$, $J > 0$, $q \geq 2$, and $N_s \geq 8$ from two scale rows; remove an independently supplied β guard from suitable clients.	Count the new lemma and any compatibility wrappers.
2	Compare rotation parameters directly	Delete Boolean premises from `rotation_eq` and `normalization_rephase`; obtain exact periodicity for arbitrary digit sizes.	The supplied production patch saves two physical lines; its main gain is the stronger contract.
3	Separate quotient intervals internally	Remove the remainder-gap witness and the explicit construction of $n < n$ in `quotient_not_lt`.	The local declaration loses two nonblank lines without a new helper.

These are deliberately three scoped recommendations. Adding a new implementation of a previously proposed generic framework would not satisfy the requested exclusion rule. Nor would checking an old proposal in a newer Lean version make its mathematical idea new.

2 Baseline, evidence, and the exclusion rule

2.1 What was inspected

The baseline is commit `f00ba53b331833c6cb64707bd90e48489e5610f5`. Its active `Lean/DiophantineConsistency` directory has 519 source files in the fetched tree; their local bytes were checked against the Git blob identities at the pin. The repository was inspected with the GitHub connection. Locally available source and toolchain material were reused only after the relevant source identities were checked or their more limited status was recorded.

The prior-proposal index contains thirty-six packets: `2026-10-26a` through `2026-10-26z`, followed by `2026-10-26za` through `2026-10-26zj`. At this snapshot, `docs/incoming` contains its README, which records their placement and retirement from the intake queue. The inspection included the primary reports, substantive notes, cumulative exclusion ledgers, and candidate-specific Lean and patch files. The closing byte-identity screen for this baseline covered all 974 Markdown, LaTeX, Lean, and patch files in the exclusion subtree: 210 Markdown files, 130 LaTeX files, 481 Lean files, and 153 patches. These include repeated snapshots as well as primary proposals. Candidate-specific searches and reading were performed across that retained text/code corpus; byte verification is not a claim that every line received equal-depth semantic review. The archive records the scope rather than claiming that every historical external document linked by the intake ledger was read.

Primary anchors are the proposal index, the adoption record, and the incoming ledger. The detailed

mechanism map is in `evidence/novelty-audit.md`.

2.2 The counterfactual baseline

The instruction to assume prior proposals applied changes the comparison. The available repository is one actual source tree; the prior packets include alternative designs, optional sketches, and interfaces not all integrated together. They cannot honestly be assigned a single measured line count. Accordingly:

1. An idea is excluded if a prior packet already supplies its mechanism or a stronger interface, whether or not the adoption log marks it implemented.
2. A candidate is retained only for an obligation that remains after granting the closest earlier interface.
3. Physical line counts refer only to the actual pinned source and the supplied candidate patch.
4. A claim of incremental savings after every counterfactual refactoring would require a separate integration, which is not asserted here.

This distinction matters especially for the new source-guard lemma. Packet b already proposed decoding the scale equations. That decoding is assumed. The new step is deriving the first port's sign from those equations and the natural-coordinate types, then deleting that sign as an input.

2.3 Examples excluded by the audit

The report does not re-propose finite support placement APIs, a common coefficient-envelope engine, generic scanner folds, deterministic witness elimination, a universal internal-induction wrapper, native rank comparison via $C + qF$, signed-prefix carry formulae, finite-chain rigidity, or source positivity from one selected digit. The recent implementation changes to the formal two-radicand combiner, doubled Pell bounds, odd-index reuse, pairing projections, impulse-based rotation, and degree-first support proofs are also treated as prior work.

Historical novelty labels are not themselves evidence of novelty. One closing audit found that packet zj retained a controller cast-normalization mechanism that packet zh had already explicitly excluded by reference to packet p. The union of the actual mechanisms is what matters.

2.4 Closing refresh: forty-three packets

Before delivery, the repository advanced to `60fcb2a69b7f1fdd4c3592ea5f164223cad50abe`, adding packets `zk` through `zq`. All three recommendations survived a fresh review of those seven packets, including their substantive notes and optional follow-on proposals. The closing receipt authenticates all 171 text/code payloads in that increment. A further sixteen substantive historical text notes were retrieved, authenticated, and checked for the same mechanisms. The full closing decisions are in `evidence/closing-novelty-audit.md`.

Packet	Additional prior mechanisms granted in the final comparison
zk	Shared Jones finalizer evaluations, affine carrier reflection, homogeneous-star enumeration, fixed-factor periods.
zl	Guarded power-cast equivalences, CRT and zero-level order endpoints, modulus transport, controller-factor positivity from coprimality.
zm	Diagonal Sidon-pair recovery, ordered-endpoint elimination, exact-order endpoint reuse, definitional depth equality.
zn	One modular-kernel certificate, power fixed-point iteration, weighted finite updates, reflected-source origin specialization.
zo	Fixed controller unit residue, exponent divisibility spectrum, carry-prefix locality between actual resets, quadratic power threshold.
zp	All-offset reflected-source identity, fixed-prime-support coprimality, arbitrary fixed-cardinality seed translation, exponent spectrum.
zq	Cyclic subset sums without coprimality, Boolean-free affine correction, fixed-period interfaces, prime-power annihilator and lift-seed APIs.

The closest new overlaps were examined explicitly. Removing the optional Boolean correction in a subset-sum construction is different from comparing two digit rotations without a Boolean digit bound. Positivity of a controller factor from coprimality does not imply the source port's sign from the first two scale rows. The new finite carry and source-update APIs likewise do not supply the targeted rotation congruence or internal quotient-interval proof.

A separate closing source check found all fourteen audited target, check, direct-dependency, and build-pin identities unchanged between the proof baseline and the closing commit. Five other production modules changed, including `ObjectPolynomial` and `SyntaxCoefficients`, which may be transitive dependencies. The compiler observations therefore remain attached to the original proof baseline. The later byte-compatibility check is not presented as a full build at the closing commit.

The expanded corpus is indexed at the forty-three-packet closing snapshot.

3 Recover the positive port guard from the source equations

3.1 The existing dependency

In `NativePellBootstrap.lean`, the declarations `source_Y_ge_two`, `source_rank_half_lt`, `source_main_and_half_bounds`, `source_base_separation`, and `source_bootstrap` take a separate hypothesis `hÎš:0<ports0`. They also take positive source coordinates and the vanishing of the literal source rows.

The source already contains enough information to recover this sign. More strongly, one source coordinate's positivity suffices for the recovery; two other signs follow from their types. Let the following names denote zero-based source coordinates:

Mathematical name	Lean expression	Type information
β	`ports0`	An integer, with no sign assumed
J	`(x0:Int)`	Nonnegative because `x0:Nat`
L	`(x24:Int)`	The one required positive coordinate
N_s	`(x16:Int)`	Nonnegative because `x16:Nat`
q	$1 + J\beta$	An integer expression

Rows 0 and 1 of `SplitCertificate59.lean` give

$$L = q^2 - 1, \qquad N_s = q(1 + L), \qquad q = 1 + J\beta. \tag{1}$$

The existing scale-decoding proposal is the natural production home for these equalities. The supplied concrete client also checks them against the actual current `sourceRows` definition, so the conclusion is not based on an unidentified abstract system.

3.2 A stronger guard-recovery theorem

Proposition 3.1 (Guard recovery)**.** *Let J, β, L, N_s be integers satisfying (1). If*

$$J \geq 0, \qquad L > 0, \qquad N_s \geq 0,$$

then

$$q \geq 2, \qquad J > 0, \qquad \beta > 0, \qquad L \geq 3, \qquad N_s \geq 8.$$

Proof. Since $1 + L > 0$ and $N_s = q(1 + L) \geq 0$, we have $q \geq 0$. If $q \leq 1$, integrality and nonnegativity give $q \in \{0, 1\}$, whereas $L = q^2 - 1 \leq 0$. Thus $q \geq 2$.

Consequently $J\beta = q - 1 > 0$. The alternative that both factors are negative is impossible because $J \geq 0$. Hence $J > 0$ and $\beta > 0$. The equation for L now gives $L \geq 3$. Finally,

$$N_s = q(1 + L) \geq 2 \cdot 4 = 8.$$

Equivalently, substituting $1 + L = q^2$ gives $N_s = q^3$, although the implementation need not expand a cubic to prove the lower bound. □

The proof avoids assuming positivity of J or N_s. Their nonnegativity is automatic in the source interpretation. It does not appeal to Pell classification, an auxiliary norm equation, a port box, or a lower bound on the native rank.

3.3 The checked concrete source client

The archive's `lean/SourceGuardClient.lean` proves the following exact consequence of the literal source rows:

$$\left(x_{24} > 0 \ \wedge \ \forall k, \ \mathrm{sourceRows}(\mathrm{ports}, x)_k = 0\right) \implies x_0 > 0 \ \wedge \ \mathrm{ports}_0 > 0 \ \wedge \ 8 \leq x_{16}. \tag{2}$$

Here $x : \mathrm{Fin}(31) \to \mathbb{N}$. Only rows 0 and 1 are used in the proof. The universally quantified source-row hypothesis matches the current API; a later interface may expose just those two equations if that has an actual caller benefit.

The second concrete theorem establishes `source_Y_ge_two_no_beta`. It assumes only positivity of coordinates 24 and 9, plus the source equations, to derive

$$2 \leq x_{16} x_9.$$

The first factor is at least eight, and the second is at least one. This supplies the existing bootstrap conclusion with fewer input assumptions. Retaining the old, weaker numerical conclusion avoids creating work for downstream clients.

3.4 How to integrate it after the earlier proposals

At the scale boundary, expose a theorem such as `source_port_guard` or add the recovered facts to the already proposed scale interface. Do not introduce another competing source record merely to store the same equations.

The first migration should remove the independent β guard from the five declarations listed above. The argument supplied to the later bootstrap theorem still includes the necessary rank and window hypotheses. This proposal says nothing that would discharge those other premises.

Additional declarations such as `source_transport_positive`, `source_native_rank_ge_twentyfour`, and `source_window_bound` also receive a β guard while already receiving a positive source. They are candidates for the same local recovery. Their separate hypotheses on K, mask ports, and mask-port upper bounds must be retained unless independently derived. The call-site inventory identifies the relevant declarations and named consumers.

There are two reasonable API strategies. A direct migration deletes the argument and updates all callers. A staged migration introduces a stronger theorem under a new name and lets the old theorem ignore its now-redundant guard while forwarding to the stronger result. The latter preserves compatibility but adds wrapper lines. This report does not count those extra declarations as compression.

3.5 Why the assumptions cannot simply disappear everywhere

The guard is recoverable because the source equations and coordinate types are present. It should remain on an abstract Pell lemma that has no such source hypothesis. Three boundary examples make the exact dependence clear:

Dropped condition	Values satisfying both scale equations	Failed conclusion
$L > 0$	$J = 1,\ \beta = 0,\ L = 0,\ N_s = 1$	$\beta > 0$
$N_s \geq 0$	$J = 1,\ \beta = -3,\ L = 3,\ N_s = -8$	$\beta > 0$
$J \geq 0$	$J = -1,\ \beta = -1,\ L = 3,\ N_s = 8$	$J > 0$ and $\beta > 0$

In the actual source, the last two conditions are guaranteed by the natural-coordinate interpretation. The positivity of coordinate 24 remains material.

3.6 Novelty boundary and expected value

Packet b already gives the scale identities. Packet f already proposes extracting `beta_pos` from an assumed `beta_min` and exposing positive source scales. Those interfaces are assumed available. Neither implies that the independently supplied port sign has already been removed from source-based bootstrap statements. The retained idea is the proof of that redundancy from the literal equations, with a smaller positivity boundary.

The immediate benefit is an acyclic dependency: source equations and natural-coordinate signs imply the port guard, and the bootstrap consumes the resulting facts. It need not depend on an independently constructed port-box certificate just to establish $\beta > 0$. This is an interface simplification, not a claim that every port-box assumption is redundant or that the final source construction has been completed.

4 Compare rotation parameters instead of proving digit bounds

4.1 What the definition actually reads

The current `NativeCyclicRotation.lean` defines

$$\mathrm{binaryRotation}(v, W, c) = \mathrm{rotate}\left(\mathrm{shiftDigits}(2^v, 2^{W \bmod v}, c), \lfloor W/v \rfloor \bmod n\right),$$

where $c : \mathrm{ZMod}(n) \to \mathbb{N}$. The displacement W is observed through exactly two values:

$$(W \bmod v, \quad (W/v : \mathrm{ZMod}(n))).$$

This dependency statement says nothing about whether c is Boolean or even bounded by the radix.

Proposition 4.1 (Exact parameter congruence)**.** *If*

$$W \bmod v = W' \bmod v, \qquad (W/v : \mathrm{ZMod}(n)) = (W'/v : \mathrm{ZMod}(n)),$$

then

$$\mathrm{binaryRotation}(v, W, c) = \mathrm{binaryRotation}(v, W', c)$$

as functions, for every natural-valued c.

Proof. Unfold `binaryRotation` and rewrite its two observed arguments. No digit arithmetic remains. □

The Lean proof is two tactic lines after the theorem header:

```
unfold binaryRotation
rw [hphase, hshift]
```

The most general checked kernel does not require `NeZeron` or $v > 0$. When either modulus is zero, it uses Lean's existing definitions; it does not infer a nontrivial finite cycle from those degenerate cases.

4.2 Exact periodicity, including partial-bit phases

The same dependency observation yields a useful parameter normal form.

Corollary 4.2 (Full bit-period reduction)**.** *For all natural v, n, W and all $c : \mathrm{ZMod}(n) \to \mathbb{N}$,*

$$\mathrm{binaryRotation}(v, W \bmod (vn), c) = \mathrm{binaryRotation}(v, W, c).$$

Consequently $W \equiv W' \pmod{vn}$ implies exact equality of the rotations, and the function of W is periodic with period vn.

Proof. Because $v \mid vn$, reducing modulo vn preserves the remainder modulo v. The division identity

$$(W \bmod (vn))/v = (W/v) \bmod n$$

preserves the second coordinate after casting to `ZModn`. Apply parameter congruence. The last statements follow from the definition of natural modular congruence. □

The implementation uses `Nat.mod_mod_of_dvd`, `Nat.mod_mul_right_div_self`, and `ZMod.natCast_mod`. This is an equality of the complete digit functions, stronger than a congruence between their evaluated words. It does not assert that the least period equals vn; particular arrays may have a smaller period.

4.3 The present rephase client

In `CurrentWordRephase2011.lean`, `rotation_eq` currently assumes `hc:foralli,ci<=1`. It then uses `rotation_zero_phase` twice to identify each binary rotation with a plain cell rotation. That route needs a digit hypothesis because it describes each rotation separately.

The theorem's actual goal only compares the two rotations. For the current displacement

$$W_s = 47 \cdot 2011 \cdot s,$$

both partial-bit phases are zero. The existing `physical_shift_eq` already transports $s = t$ in `ZModN` to equality of the physical quotient shifts in `ZMod(N*2011)`. Parameter congruence therefore proves the desired equality for arbitrary digits.

Two dependent changes follow. First, delete `hc` from `rotation_eq`. Second, delete `hd:forallj,dummyj<=1` from `normalization_rephase`, where it is used only to establish `constructed_boolean` for this comparison. Update the one production call and the focused check call recorded in the patch. The existing `normalization_of_rotation_eq` derives the new scalar transport from exact equality of rotations, so it still transfers the same carry and digit arrays.

4.4 The distinction that prevents an unsound shortcut

For unbounded digits, a zero-phase binary rotation need not equal the original digit function. With $v = 3$, $n = 5$, and $c(i) = 8$ everywhere,

$$\mathrm{binaryRotation}(3, 0, c)(i) = 8 \bmod 8 + \lfloor 8/8 \rfloor = 1,$$

whereas $c(i) = 8$. The checked example in `RotationCongruence.lean` records this failure of the stronger, incorrect simplification.

Thus the proposal compares two equally normalized expressions; it does not delete the guard from `rotation_zero_phase`. Nor does it delete `hd` from `marked_packing_rephase` or `packing_rephase`: those later theorems still use Boolean bounds for masks and scalar construction. The ordinary temporal multipliers can change when the stride changes, even though the rotations agree. The proof must continue deriving the appropriate transport congruence rather than claiming equality of ordinary multipliers.

4.5 Novelty boundary and accounting

Earlier packets cover half-radix splitting, point-mass convolution, sparse arbitrary-phase support, and equality of evaluated words. Those are assumed. The residual claim is equality from the two coordinates actually read by the definition, and the removal of two newly introduced Boolean premises from the rephase interface.

The local production patch changes the main rephase module from 122 to 120 physical lines; its check module remains 80 lines. The generic exposition kernels add lines if installed in full. The sensible integration choice is either the minimal direct proof in the current module, or a small shared congruence theorem when another actual caller appears. The periodicity result is a useful optional extension of the same idea, not a fourth independent proposal and not a basis for extrapolated line savings.

5 Prove internal division uniqueness by interval separation

5.1 The exact residual proof

`ObjectDivision.lean` works over `NaturalH`, the naturals of an arbitrary model satisfying the displayed set-theoretic axioms. Its relation is

$$\mathrm{DivisionRel}(q, r, n, d) \quad \Longleftrightarrow \quad n = qd + r \ \wedge \ r < d.$$

The private lemma `quotient_not_lt` compares two such certificates. Currently, assuming $q < q'$, it constructs witnesses for both

$$q' = q + s + 1, \qquad d = r + t + 1,$$

expands a ring identity, and constructs an explicit successor witness for $n < n$.

The second witness is avoidable. The remainder bound already expresses the strict interval information needed by the proof.

5.2 The interval argument

Proposition 5.1 (Separation of quotient intervals)**.** *In the internal natural semiring, suppose* $q' = q + s + 1$ *and* $r < d$*. Then for every internal natural* r'*,*

$$qd + r < q'd + r'.$$

Proof. Use the chain

$$qd + r < qd + d \le q'd \le q'd + r'.$$

The first step is translation invariance of strict additive order. For the second step, the nonnegative additive gap is sd, since

$$(qd + d) + sd = (q + s + 1)d = q'd.$$

For the third step, the additive gap is r'. The existing characterization of internal order by an additive witness proves both weak inequalities. □

If two division certificates have the same dividend, the chain contradicts their equality. Applying the argument in both directions gives $q = q'$, after which additive cancellation gives $r = r'$.

5.3 What the delivered replacement removes

The supplied local patch retains one witness s for $q < q'$ and one short ring normalization for the middle inequality. It removes the separate remainder-gap witness t and the specially manufactured witness for $n < n$. It does not claim to make the proof wholly witness-free.

The abstract kernel `DivisionInterval.lean` is checked for a commutative semiring with linear order, ordered cancellative addition, and the displayed additive-order laws. Multiplication monotonicity is established only at the needed comparison using sd. No new global ordered-multiplication instance is needed by this candidate.

If the previously proposed ordered-semiring API has already been installed, the same middle step can be expressed directly using $(q + 1) \le q'$ and nonnegative multiplication. That is an integration choice using an assumed earlier API. The present packet's new mathematical content is the interval separation of the two division certificates.

5.4 Why host-natural division is not a replacement

One must not silently use `Nat.div_mod_unique` on arbitrary `NaturalH`. An arbitrary model's natural numbers need not all be represented by external standard numerals. The earlier packet k correctly says that its host-`Nat` certificate theorem does not replace internal `ObjectDivision` statements without a separate internal semantic argument.

The interval proof supplies that remaining local argument without any external induction on the model carrier. The internal-induction proof of division existence is untouched. The convention for divisor zero is untouched. A division certificate with $r < d$ already rules out $d = 0$, but the uniqueness proof need not impose a new explicit positive-divisor parameter.

5.5 Novelty boundary and measured value

The closing code audit inspected packet zf's proposed `ObjectDivision.lean`. It edits `successor_step` and `division_exists`, while leaving the old two-witness `quotient_not_lt` argument unchanged. This proposal is therefore about a different, still-present obligation.

The local declaration decreases from eleven to nine nonblank lines, with no new production helper or import. A standalone reusable kernel increases total source size if added for only this one caller. The recommendation is to use the direct local interval proof, or reuse an existing order API, and to justify the change by the simpler invariant. It is a lower-priority cleanup than removing redundant theorem hypotheses.

6 Validation and interpretation of the artifacts

The checked kernels use Lean 4.32.0. The source-row client was freshly elaborated against the production `SplitCertificate59` import; the imported module's complete source is byte-identical to the reviewed snapshot. The current source of both `NativeCyclicConvolution` and `NativeCyclicRotation` was freshly compiled before the final rotation checks. Their direct project dependency `Diophantine.Paper1982.Master2` was authenticated against the pinned Git blob. Deeper imported dependencies were supplied by the available cache.

Artifact	Status	Exact scope
`SourceGuardRecovery.lean`	Checked, exit 0	Two generic integer guard and scale theorems.
`SourceGuardClient.lean`	Checked, exit 0	Two clients of the actual current source-row definition.
`DivisionInterval.lean`	Checked, exit 0	Abstract interval argument, uniqueness theorem, exact local proof skeleton, and ordered-interface variant.
`RotationCongruence.lean`	Checked, exit 0	Parameter congruence, whole-field specialization, the actual numerical displacement pattern, and digit-boundary examples.
`RotationPeriod.lean`	Checked, exit 0	Reduction of displacement modulo the bit period, modular congruence, and periodicity.

Artifact	Status	Exact scope
Rephase candidate patch	Application checked	Two Boolean premises removed; complete rephase module and downstream clients not rebuilt.
Internal division patch	Application checked	One private theorem changed; complete object module not rebuilt.
Beta-input migration	Proposed	Concrete stronger client checked; signature migration across bootstrap consumers not performed.

The fifteen printed axiom reports for the checked endpoints use only `propext`, `Classical.choice`, and `Quot.sound`, or subsets thereof. No full repository build or aggregate production speedup is claimed.

6.1 What the checks establish

A successful Lean invocation verifies the supplied statements against the imported declarations in that invocation. The accompanying axiom reports distinguish ordinary Lean foundations from a placeholder or an added assumption. Kernel examples and negative controls check the intended guard boundaries, rather than mirroring a proof's syntax.

The source-row client is especially useful evidence: its two `change` statements must match the actual polynomial rows, and the linear-combination steps must produce the displayed scale equalities. A general numerical lemma alone would not establish this source binding.

The source manifests and patch-application checks serve a different purpose. They pin the files on which the proposals were based and show where a candidate applies; they do not themselves establish Lean elaboration. Conversely, a checked standalone kernel is not evidence that every updated repository caller has been migrated. The validation table keeps these scopes separate.

6.2 No claimed end-to-end consistency advance

These changes do not construct the outstanding rank-calibrated temporal exponent, the remaining positive Pell witnesses, or a complete solution of the final source system. They simplify arguments already used around those obligations. In particular, proving that an existing positive source forces $\beta > 0$ is a conditional theorem, not a construction of that positive source.

7 Integration plan and stopping criteria

7.1 Stage 1: source guard recovery

Add the guard-recovery theorem beside the existing scale interface. Verify the concrete source interpretation once. Introduce a source-facing theorem with fewer hypotheses and preserve the old wrapper if required by current consumers. Then migrate the five direct bootstrap clients as one reviewable change. A later pass may remove redundant β inputs from other source-facing theorems, but it should explicitly retain all other port and rank assumptions.

The acceptance criterion is a checked statement comparison: all original conclusions remain available, and the only removed inputs are consequences of already present source equations and signs. Count the helper and compatibility wrappers. There is no need to reorganize the entire source system to obtain this benefit.

7.2 Stage 2: rotation congruence

Apply the small rephase patch and run its focused checks and direct consumers. Keep the arbitrary-digit regression example, because it prevents a future rewrite from replacing exact parameter equality by an invalid unrestricted plain-rotation law. Install the complete periodicity API only if a second displacement-normalization client benefits from it.

The acceptance criterion is preservation of the same source, carry array, and normalized digit function. Exact rotation equality is the transport premise. Equality of ordinary multipliers is neither needed nor generally true.

7.3 Stage 3: interval uniqueness

Replace the private proof locally. Use the prior ordered-semiring interface if it is already the preferred model API; otherwise use the one-gap version in the supplied candidate. Recheck division uniqueness, the quotient/remainder specification, and the existing external-numeral comparison.

The acceptance criterion is that the proof remains valid for arbitrary internal model naturals. A shorter proof that assumes all such naturals are standard would change the theorem's mathematical strength and must be rejected.

8 Archive guide and reproducibility

Location	Purpose
`report/report.tex`	Complete LaTeX source of this report.
`report/report.pdf`	Compiled report, with rendered-page quality checks.
`lean/`	Checked guard, rotation, and interval kernels; concrete source-row client; boundary examples.
`patches/`	Scoped rephase and internal-division candidate changes.
`evidence/`	Source identities, compiler output, validation scopes, call-site inventory, nonduplication audit, and independent review.
`scripts/`	Focused replay, source-manifest verification, and report build commands.
`SHA256SUMS`	Integrity manifest for the deliverable files.

The replay script expects a checkout at the documented commit with its Lean 4.32.0 Lake environment available. It checks source identities and compiles the supplied kernels in a separate output directory. It does not silently modify the user's checkout. Full candidate integration remains a distinct operation, as recorded by the validation status.

Selected sources and nearest prior proposals

All repository links below refer to the reviewed commit. The archive's source manifest identifies the exact bytes used.

1. NativePellBootstrap: source-facing positive-port inputs, scale bounds, and bootstrap consumers.
2. SplitCertificate59: the actual first two source rows.
3. NativeCyclicRotation: exact binary-rotation definition and evaluated-word transport.
4. CurrentWordRephase2011: rephasing of the same source, carries, and digit arrays.
5. ObjectDivision: arbitrary-model division existence and uniqueness.
6. Packet b, report: previously proposed scale interfaces, granted as baseline here.
7. Packet f, architecture: positivity accessors and extraction from already assumed port bounds.
8. Packet k, certificate notes: explicit distinction between host-natural and internal-model division.
9. Packet zf, report: internal successor and arithmetic maintenance; its division candidate leaves the targeted uniqueness proof unchanged.
10. Packet zj, novelty inventory, and packet zh, exclusion ledger: useful indexing aids, checked against actual mechanisms rather than adopted without review.